

Analyse de la Diffusion Large de Messages RSA

Ayman LAABADI - Walid MAIOUTE - Samy LAKHNATI - Ali BRAIKI
Encadré par : Pr. Pierre-Vincent Koseleff

August 29, 2026

Abstract

Ce rapport étudie la vulnérabilité du schéma RSA dans le contexte de la diffusion large d'un même message vers plusieurs destinataires. Nous montrons que l'utilisation d'un exposant public commun et de modules premiers entre eux permet à un adversaire de reconstruire le message en clair dès que certaines conditions arithmétiques sont satisfaites, notamment lorsque le nombre de destinataires interceptés dépasse l'exposant public. Une analyse quantitative met en évidence les seuils de sécurité associés, complétée par des simulations illustrant les cas favorables et défavorables à l'attaque. Enfin, nous présentons les théorèmes fondamentaux de Håstad et de Coppersmith, qui généralisent cette vulnérabilité — y compris en présence de padding linéaire — et garantissent l'existence d'algorithmes efficaces de récupération du message.

1 Énonciation du problème

Considérons un scénario de *diffusion large* dans lequel Alice souhaite transmettre un même message m à un ensemble de r destinataires. Chaque destinataire i possède une clé publique RSA (n_i, e) , où tous les modules n_i sont **premiers deux à deux** :

$$\gcd(n_i, n_j) = 1 \quad \text{pour } i \neq j.$$

Chaque destinataire reçoit alors un chiffré

$$c_i \equiv m^e \pmod{n_i}, \quad i = 1, \dots, r.$$

En posant $N = n_1 n_2 \cdots n_r$, le théorème des restes chinois garantit l'existence d'une unique solution $X \in [0, N[$ au système suivant :

$$X \equiv c_i \pmod{n_i} \quad \forall i.$$

Par construction, puisque $c_i \equiv m^e \pmod{n_i}$, il s'ensuit nécessairement que

$$X \equiv m^e \pmod{N}.$$

L'idée fondamentale de l'attaque repose alors sur le passage d'une congruence à une égalité dans $\mathbb{Z}$. Lorsque m^e est strictement inférieur à N , la congruence devient une égalité exacte :

$$X = m^e.$$

Dans ce cas, récupérer le message revient simplement à calculer la racine entière e -ième de X .

Théorème 1 (Condition critique de déchiffrement). *L'attaque de diffusion est garantie lorsque*

$$m^e < N = \prod_{i=1}^r n_i.$$

Dans ce cas, l'assemblage par le théorème des restes chinois donne exactement $X = m^e$, et le message peut être récupéré par extraction de la racine entière :

$$m = \sqrt[e]{X}.$$

Dans un système RSA correctement dimensionné, le message m est toujours strictement inférieur au module du destinataire. Si l'on note par ℓ_m la taille en bits du message et par ℓ_n celle du plus petit module $n_{\min}$, on a typiquement

$$|m| \approx 2^{\ell_m}, \quad |n_{\min}| \approx 2^{\ell_n},$$

avec par exemple $\ell_n = 2048$ bits dans la pratique moderne, contre environ $\ell_m = 256$ bits pour un bloc de message.

Le risque cryptographique ne provient donc pas de la condition $m < n_{\min}$, toujours vérifiée, mais de la possibilité que la puissance m^e demeure inférieure au produit N . Cette condition dépend directement du rapport entre le nombre de destinataires r et l'exposant public e .

Corollaire 2 (Condition sur le nombre de destinataires). *Si le nombre de destinataires interceptés satisfait $r \geq e$, alors l'attaque est réalisable de manière déterministe.*

Proof. Lorsque $r \geq e$, on a

$$N = n_1 n_2 \cdots n_r \geq n_{\min}^e.$$

Puisque par hypothèse $m < n_{\min}$, il s'ensuit que $m^e < n_{\min}^e \leq N$, ce qui satisfait la condition critique précédente. L'extraction de la racine entière fournit alors immédiatement :

$$m = \lfloor \sqrt[e]{N} \rfloor.$$

□

Ainsi, l'inversion de RSA sans clé privée se réduit à un simple problème numérique : le calcul d'une racine entière.

2 Méthodes de génération de modules RSA

2.1 Problématique

La problématique de ce travail est donc la suivante :

- *Comment générer rapidement de grands nombres premiers de taille donnée (en bits) pour la création de clés publiques, en s'appuyant sur le test de Miller–Rabin ?*
- *Quelle est la probabilité d'erreur réelle de ce test en pratique, et dans quelle mesure est-elle cohérente avec la borne théorique 4^{-t} donnée par le théorème de Rabin ?*
- *Combien d'essais faut-il en moyenne pour obtenir un nombre premier de ℓ bits, et cette moyenne est-elle conforme aux prédictions du théorème des nombres premiers ?*

L'objectif de ce rapport est d'étudier ces questions à la fois sur le plan théorique (démonstration des résultats de Rabin et estimation du nombre moyen d'essais nécessaires) et sur le plan expérimental (simulations numériques et mesures de performance), afin de mieux comprendre et justifier l'utilisation de Miller–Rabin pour la génération de grands nombres premiers cryptographiques.

2.2 Fondements théoriques

Proposition 3 (Cas p premier impair). *Soit p un nombre premier impair. On écrit $p - 1 = 2^s t$ avec t impair. Alors pour tout $a \in (\mathbb{Z}/p\mathbb{Z})^\times$, on a :*

$$a^t = 1 \quad \text{ou} \quad \exists j \in \{0, 1, \dots, s-1\} : a^{2^j t} = -1.$$

Proof. Soit $a \in (\mathbb{Z}/p\mathbb{Z})^\times$ et soit r l'ordre de a dans $(\mathbb{Z}/p\mathbb{Z})^\times$. On sait que $r \mid p - 1 = 2^s t$.

Cas 1 : r est impair.

Alors $r \mid t$ (car t est la partie impaire de $p - 1$), donc $a^t = 1$.

Cas 2 : r est pair.

On écrit $r = 2^{r'} u$ avec u impair et $r' \geq 1$. Alors $u \mid t$ donc $a^t = (a^u)^{t/u}$. Comme a^u a un ordre qui est une puissance de 2 (précisément $2^{r'}$), on a $(a^u)^{2^{r'-1}} = -1$ dans $\mathbb{Z}/p\mathbb{Z}$ (car l'ordre de a^u est exactement $2^{r'}$ et dans un corps, la seule racine carrée de 1 différente de 1 est -1).

Donc $a^{2^{r'-1}u} = -1$. Comme $u \mid t$, on peut écrire $t = u \cdot \ell$ avec ℓ impair, d'où

$$a^{2^{r'-1}t} = (a^{2^{r'-1}u})^\ell = (-1)^\ell = -1$$

(car ℓ est impair). Ainsi il existe $j = r' - 1 \in \{0, 1, \dots, s-1\}$ tel que $a^{2^j t} = -1$. □

Théorème 4 (Rabin – cas n impair composé). *Soit n un entier impair composé. On écrit $n - 1 = 2^s t$ avec t impair. Soit la décomposition en facteurs premiers*

$$n = \prod_{i=1}^m p_i^{k_i}$$

avec $m \geq 2$ ou bien $k_1 \geq 2$. Pour chaque i , notons $\varphi_i = \varphi(p_i^{k_i}) = p_i^{k_i-1}(p_i - 1)$ et écrivons

$$\varphi_i = 2^{v_i} u_i$$

avec u_i impair.

Définissons les ensembles suivants dans $(\mathbb{Z}/n\mathbb{Z})^\times$:

$$\begin{aligned} M_0 &= \{x \in (\mathbb{Z}/n\mathbb{Z})^\times : x^t = 1\}, \\ M'_j &= \{x \in (\mathbb{Z}/n\mathbb{Z})^\times : x^{2^j t} = -1\} \quad \text{pour } j \in \{0, 1, \dots, s-1\}, \\ M &= M_0 \cup M'_0 \cup M'_1 \cup \dots \cup M'_{s-1}. \end{aligned}$$

Posons également

$$w_i = \gcd(t, u_i) \quad \text{et} \quad r = \min(s, v_1, \dots, v_m).$$

Alors on a les majorations suivantes :

$$|M| \leq \frac{2^{m(r-1)+1} - 1}{2^m - 1} \prod_{i=1}^m w_i \leq 2^{m(r-1)+1} \prod_{i=1}^m w_i.$$

En particulier, si n est impair composé, alors

$$|M| \leq \frac{1}{4} \varphi(n).$$

Proof. Par le théorème chinois des restes, on a l'isomorphisme

$$(\mathbb{Z}/n\mathbb{Z})^\times \simeq \prod_{i=1}^m (\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times.$$

Chaque groupe $(\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times$ est cyclique d'ordre $\varphi_i = 2^{v_i} u_i$.

Calcul de $|M_0|$. Un élément $x \in (\mathbb{Z}/n\mathbb{Z})^\times$ vérifie $x^t = 1$ si et seulement si, pour chaque composante $x_i \in (\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times$, on a $x_i^t = 1$. L'ordre de x_i divise φ_i , donc le nombre de solutions dans $(\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times$ est

$$\gcd(t, \varphi_i).$$

Or t est impair et $\varphi_i = 2^{v_i} u_i$, donc

$$\gcd(t, \varphi_i) = \gcd(t, u_i) = w_i.$$

Par conséquent,

$$|M_0| = \prod_{i=1}^m w_i.$$

Calcul de $|M'_j|$ pour $0 \leq j \leq s-1$. Un élément $x \in (\mathbb{Z}/n\mathbb{Z})^\times$ vérifie $x^{2^j t} = -1$ si et seulement si, pour chaque composante x_i , on a $x_i^{2^j t} = -1$ dans $(\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times$.

Condition nécessaire : Pour qu'il existe $x_i \in (\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times$ tel que $x_i^{2^j t} = -1$, il faut que -1 soit une puissance $(2^j t)$ -ième dans $(\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times$. L'ordre de -1 dans ce groupe cyclique divise $\varphi_i/2 = 2^{v_i-1}u_i$. En pratique, si $j \geq v_i$, alors $2^j t$ est divisible par 2^{v_i} donc $x_i^{2^j t}$ est nécessairement 1, et ne peut être -1 . Donc $|M'_j| = 0$ si $j \geq r$ (car $r = \min(s, v_1, \dots, v_m)$).

Cas $0 \leq j < r$: Dans ce cas, $j < v_i$ pour tout i , donc $2^j t$ n'est pas divisible par 2^{v_i} . Le nombre de solutions x_i vérifiant $x_i^{2^j t} = -1$ dans $(\mathbb{Z}/p_i^{k_i}\mathbb{Z})^\times$ est

$$\gcd(2^j t, \varphi_i) = \gcd(2^j t, 2^{v_i} u_i) = 2^j \gcd(t, u_i) = 2^j w_i$$

(car $j < v_i$ et t est impair).

Donc

$$|M'_j| = \prod_{i=1}^m 2^j w_i = 2^{mj} \prod_{i=1}^m w_i \quad \text{pour } 0 \leq j < r.$$

Majoration de $|M|$. Les ensembles $M_0, M'_0, M'_1, \dots, M'_{s-1}$ sont disjoints. Donc

$$|M| = |M_0| + \sum_{j=0}^{s-1} |M'_j|.$$

Or $|M'_j| = 0$ pour $j \geq r$, donc

$$|M| = |M_0| + \sum_{j=0}^{r-1} |M'_j| = \prod_{i=1}^m w_i + \sum_{j=0}^{r-1} 2^{mj} \prod_{i=1}^m w_i = \left(1 + \sum_{j=0}^{r-1} 2^{mj}\right) \prod_{i=1}^m w_i.$$

En sommant, on obtient :

$$|M| = \left(1 + 1 + 2^m + 2^{2m} + \dots + 2^{m(r-1)}\right) \prod_{i=1}^m w_i.$$

Ainsi,

$$\begin{aligned} |M| &\leq \frac{2^{m(r-1)+1} - 1}{2^m - 1} \prod_{i=1}^m w_i \leq 2^{m(r-1)+1} \prod_{i=1}^m w_i. \\ \frac{|M|}{\varphi(n)} &\leq \frac{2^{m(r-1)+1} \prod_{i=1}^m w_i}{2^{\sum_{i=1}^m v_i} \prod_{i=1}^m u_i} = \frac{1}{2^{(\sum_{i=1}^m v_i) - m(r-1) - 1}} \cdot \frac{\prod_{i=1}^m w_i}{\prod_{i=1}^m u_i} \end{aligned}$$

Par conséquent,

$$|M| \leq \frac{1}{4} \varphi(n),$$

ce qui signifie qu'au moins $\frac{3}{4}$ des éléments de $(\mathbb{Z}/n\mathbb{Z})^\times$ ne satisfont pas les conditions du test de Miller-Rabin, et peuvent donc servir de témoins de la compositeness de n . $\square$

2.3 Implémentation du test de Miller-Rabin

2.3.1 Description de l'algorithme

Le test de Miller-Rabin est un test de primalité probabiliste qui repose sur le théorème précédent. Pour tester si un nombre impair n est premier, on procède en deux étapes :

1. **Décomposition :** On écrit $n - 1 = 2^s \cdot d$ où d est impair.
2. **Test avec une base aléatoire :** On choisit un entier a aléatoire dans $\{2, 3, \dots, n-2\}$ et on vérifie si :
 - soit $a^d \equiv 1 \pmod{n}$,

- soit il existe $j \in \{0, 1, \dots, s-1\}$ tel que $a^{2^j d} \equiv -1 \pmod{n}$.

Si aucune de ces conditions n'est vérifiée, alors n est **composé** (avec certitude). Si l'une des conditions est vérifiée, alors n est **probablement premier**.

En répétant le test avec t bases aléatoires différentes, la probabilité qu'un nombre composé soit déclaré premier est au plus 4^{-t} , d'après le théorème de Rabin.

2.3.2 Code Python commenté

Listing 1: Décomposition de $n-1$ en $2^s \cdot d$

```

1 from random import randrange
2
3 # 1) Décomposition  $n-1 = 2^s \cdot d$ 
4 def decomposition_nm1(n):
5     s = (n-1).valuation(2) # exposant de 2 dans  $n-1$ 
6     d = (n-1) // (2**s)    # d impair
7     return s, d

```

Explication : La fonction `valuation(2)` calcule la plus grande puissance de 2 qui divise $n-1$, c'est-à-dire l'exposant s dans la décomposition $n-1 = 2^s \cdot d$. Ensuite, on calcule $d = (n-1)/2^s$.

Listing 2: Un tour du test de Miller-Rabin avec une base a

```

1 # 2) Un tour de Miller-Rabin (une base  $a$ )
2 def miller_rabin_round(a, n):
3     s, d = decomposition_nm1(n)
4     x = power_mod(a, d, n) #  $a^d \pmod{n}$ 
5
6     # cas 1 :  $x = 1$  ou  $-1 \pmod{n}$ 
7     if x == 1 or x == n-1:
8         return True
9
10    # cas 2 : on itère  $x \leftarrow x^2 \pmod{n}$ 
11    for _ in range(s-1):
12        x = power_mod(x, 2, n) #  $x = x^2 \pmod{n}$ 
13        if x == n-1:
14            return True
15
16    # jamais tombe sur 1 ou  $n-1 \rightarrow a$  est témoin  $\rightarrow n$  est composé
17    return False

```

Explication :

- On calcule d'abord $x = a^d \pmod{n}$.
- **Cas 1 :** Si $x = 1$ ou $x = -1 \equiv n-1 \pmod{n}$, alors a satisfait les conditions de primalité, et on retourne `True`.
- **Cas 2 :** Sinon, on itère $s-1$ fois en calculant $x \leftarrow x^2 \pmod{n}$, ce qui correspond à calculer $a^{2^j d} \pmod{n}$ pour $j = 1, 2, \dots, s-1$. Si à un moment $x = -1 \pmod{n}$, on retourne `True`.
- Si aucune des conditions n'est vérifiée après $s-1$ itérations, alors a est un *témoin* de la compositeness de n , et on retourne `False`.

Listing 3: Test complet de Miller-Rabin avec t itérations

```

1 # 3) Test complet Miller-Rabin avec  $t$  itérations
2 def miller_rabin(n, t=10):
3     if n < 2:
4         return False
5     if n == 2:
6         return True
7     if n % 2 == 0:
8         return False
9
10    small_primes = [3, 5, 7, 11, 13, 17, 19, 23, 29]
11    for p in small_primes:

```

```

12     if n == p:
13         return True
14     if n % p == 0:
15         return False
16
17     for _ in range(t):
18         a = randrange(2, n-1) # base aleatoire dans [2, n-2]
19         if not miller_rabin_round(a, n):
20             return False # temoin trouve -> compose
21
22     return True # probablement premier

```

Explication :

- On traite d'abord les cas triviaux : $n < 2$ (non premier), $n = 2$ (premier), n pair (non premier sauf 2).
- On teste ensuite si n est divisible par de petits nombres premiers (3, 5, 7, ..., 29). Si n est égal à l'un de ces nombres, il est premier. Si n est divisible par l'un d'eux, il est composé.
- Ensuite, on effectue t tours du test de Miller-Rabin avec des bases aléatoires $a \in \{2, 3, \dots, n-2\}$.
- Si l'un des tours retourne **False**, alors n est **composé avec certitude**.
- Si tous les tours retournent **True**, alors n est **probablement premier** avec une probabilité d'erreur au plus 4^{-t} .

2.3.3 Simulation : génération de grands nombres premiers

Listing 4: Génération de nombres premiers de 1024 bits

```

1 bits = 1024
2 primes = []
3 nb_tirages = 20000
4
5 for i in range(nb_tirages):
6     p = randrange(2**(bits-1), 2**bits) # nombre de 1024 bits
7     p |= 1 # le rendre impair
8
9     if miller_rabin(p, t): # t iterations
10         primes.append(p)
11         print(f"[p] *") # affichage pour eviter trop d'informations
12
13 # Exemple de nombre premier trouve (1024 bits) :
14 # 118188486132916842834119683840344039445077855647302733803805235...

```

Explication :

- On génère 20 000 nombres aléatoires de 1024 bits.
- Pour chaque nombre p , on s'assure qu'il est impair (en mettant le bit de poids faible à 1 avec $p |= 1$).
- On teste ensuite si p est probablement premier avec le test de Miller-Rabin.
- Si le test retourne **True**, on ajoute p à la liste des nombres premiers trouvés.
- En pratique, pour une utilisation cryptographique, on devrait utiliser $t \geq 40$ pour garantir une probabilité d'erreur inférieure à 2^{-80} .

2.4 Analyse détaillée de la complexité

2.4.1 Complexité d'un tour de Miller-Rabin

La fonction `miller_rabin_round(a, n)` effectue les opérations suivantes :

1. **Décomposition** : $O((\log n)^2)$.
2. **Calcul de $x = a^d \bmod n$** : $O((\log n)^3)$ (exponentiation modulaire).

3. **Test** $x = 1$ ou $x = n - 1$: $O(\log n)$ (comparaisons).

4. **Boucle sur $s - 1$ itérations :**

- À chaque itération : calcul de $x \leftarrow x^2 \bmod n$ (une multiplication modulaire : $O((\log n)^2)$) et un test de comparaison ($O(\log n)$).
- Nombre d'itérations : $s - 1 \leq \log_2(n - 1) = O(\log n)$.
- Complexité de la boucle : $O(\log n) \times O((\log n)^2) = O((\log n)^3)$.

Complexité totale d'un tour :

$$O((\log n)^2) + O((\log n)^3) + O((\log n)^3) = O((\log n)^3).$$

2.4.2 Complexité du test complet

La fonction `miller_rabin(n, t)` effectue les opérations suivantes :

- Génération d'un nombre aléatoire a : $O(\log n)$.
- Appel à `miller_rabin_round(a, n)` : $O((\log n)^3)$.
- Nombre d'itérations : t .
- Complexité de la boucle : $O(t \cdot (\log n)^3)$.

2.4.3 Complexité de la génération de nombres premiers

Pour générer un nombre premier de ℓ bits, on procède comme suit :

1. Tirer un nombre aléatoire impair p de ℓ bits.
2. Tester si p est premier avec Miller-Rabin (t itérations).
3. Si p n'est pas premier, recommencer.

2.4.4 Nombre d'essais attendus : analyse détaillée

Théorème des nombres premiers : Le nombre de nombres premiers inférieurs ou égaux à x , noté $\pi(x)$, vérifie :

$$\pi(x) \sim \frac{x}{\ln(x)} \quad \text{quand } x \rightarrow \infty.$$

Cela signifie que la **densité asymptotique** des nombres premiers parmi tous les entiers positifs autour de x est :

$$\frac{\pi(x)}{x} \sim \frac{1}{\ln(x)}.$$

En d'autres termes, la probabilité qu'un entier pris au hasard autour de x soit premier est approximativement $\frac{1}{\ln(x)}$.

Cas des nombres impairs : Lorsqu'on génère des nombres premiers cryptographiques, on se restreint aux nombres **impairs** (car tout nombre pair supérieur à 2 n'est pas premier). Parmi les nombres impairs autour de x , la densité des nombres premiers est environ **deux fois plus élevée** que dans l'ensemble de tous les entiers, car on élimine d'emblée la moitié des candidats (les nombres pairs).

Ainsi, la probabilité qu'un nombre impair pris au hasard autour de x soit premier est approximativement :

$$p = \frac{2}{\ln(x)}.$$

Nombre moyen de tirages : Si la probabilité de succès (tirer un nombre premier) à chaque essai est $p = \frac{2}{\ln(x)}$, alors le nombre moyen d'essais avant d'obtenir un succès suit une loi géométrique de paramètre p , dont l'espérance est :

$$\mathbb{E}[\text{nombre d'essais}] = \frac{1}{p} = \frac{\ln(x)}{2}.$$

Application numérique : Pour des nombres de $\ell = 1024$ bits, on a $x \approx 2^\ell = 2^{1024}$. Le logarithme naturel de x est :

$$\ln(x) = \ln(2^{1024}) = 1024 \ln(2) \approx 1024 \times 0,693 \approx 710.$$

Donc, le nombre moyen de tirages nécessaires pour trouver un nombre premier de 1024 bits parmi les nombres impairs est :

$$\frac{\ln(x)}{2} \approx \frac{710}{2} = 355 \text{ essais en moyenne.}$$

Remarque 1. En pratique, l'utilisation d'un pré-filtrage (test de divisibilité par de petits nombres premiers comme 3, 5, 7, etc.) permet de réduire encore ce nombre d'essais, car on élimine rapidement les candidats composés évidents avant d'appliquer Miller-Rabin.

2.4.5 Validation expérimentale

Nous avons réalisé une simulation pour vérifier empiriquement la validité de cette prédiction théorique. Pour différentes tailles de bits, nous avons généré un grand nombre de candidats impairs et compté combien étaient effectivement premiers.

Résultats de la simulation :

Bits	Premiers trouvés	Proba empirique	Proba théorique $\frac{2}{\ln(2^\ell)}$	Essais/prime (théorique)
64	940 / 20000	0.047	0.0451	22.2
128	453 / 20000	0.0227	0.0225	44.4
256	227 / 20000	0.0114	0.0113	88.7
512	112 / 20000	0.0056	0.0056	177.4

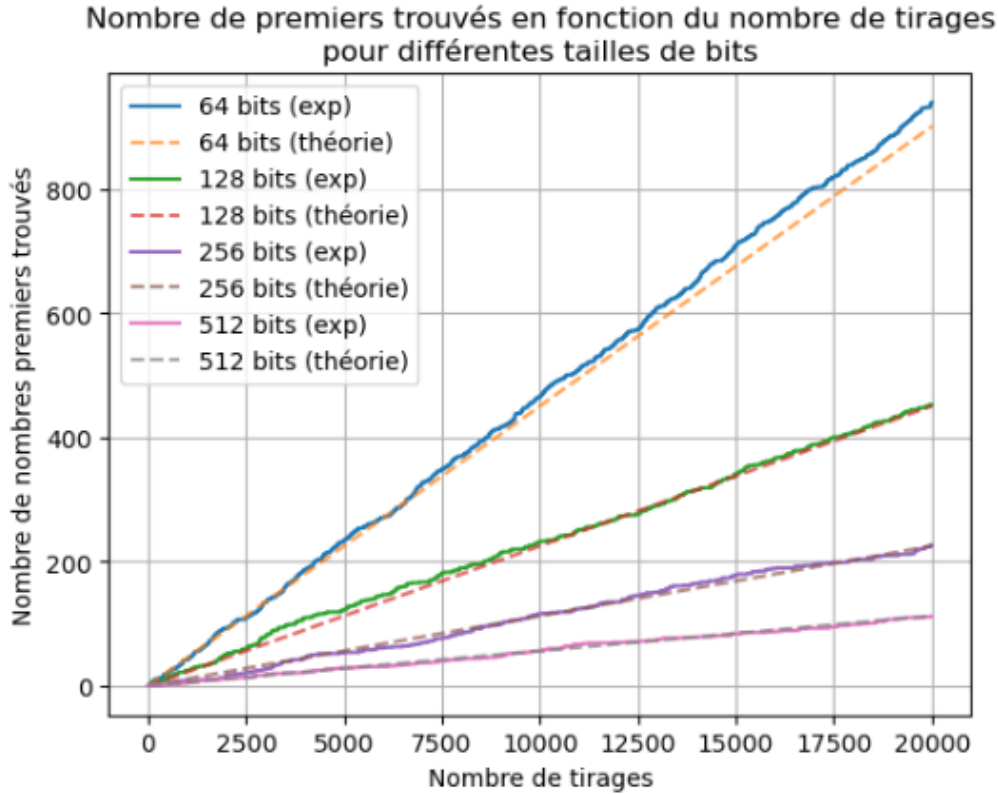


Figure 1: Nombre de nombres premiers trouvés en fonction du nombre de tirages pour différentes tailles de bits. Les courbes expérimentales (traits pleins) correspondent bien aux prédictions théoriques (traits pointillés), validant la formule $\mathbb{E}[\text{essais}] = \frac{\ln(2^\ell)}{2}$ pour la génération de nombres premiers cryptographiques.

Les écarts observés entre théorie et pratique sont minimes et dus aux fluctuations statistiques inhérentes à tout échantillonnage fini.

2.4.6 Complexité totale de la génération

Chaque essai nécessite :

- Un test de Miller-Rabin avec t itérations : $O(t \cdot (\log n)^3)$.
- Le pré-filtrage par petits premiers : $O((\log n)^2)$ (négligeable devant le coût de Miller-Rabin).

En moyenne, on effectue $\frac{\ln(n)}{2}$ essais. Donc la complexité totale attendue est :

$$\text{Complexité totale} = O\left(\frac{\ln(n)}{2} \times t \cdot (\log n)^3\right) = O(t \cdot (\log n)^4).$$

Application numérique : Pour $\ell = 1024$ bits et $t = 10$ itérations :

$$\text{Nombre d'essais} \approx 355, \quad \text{Complexité} \approx 355 \times 10 \times 1024^3 \approx 4 \times 10^{12} \text{ opérations bit.}$$

En pratique, sur un processeur moderne, cela correspond à quelques secondes pour générer un nombre premier de 1024 bits.

2.5 Validation expérimentale de la borne théorique

Le théorème de Rabin garantit que la distribution des nombres impairs composés qui vérifient le test est inférieure à $1/4$, et pour t itérations on a

$$P(\text{erreur}) \leq \left(\frac{1}{4}\right)^t = 4^{-t}.$$

Nous avons réalisé une simulation empirique pour vérifier cette borne théorique.

2.5.1 Description de l'expérience

Afin d'évaluer empiriquement la probabilité d'erreur du test de Miller–Rabin, nous avons mis en place l'expérience suivante.

Test de primalité de référence. Pour pouvoir savoir si Miller–Rabin se trompe ou non, il nous faut d'abord un test de primalité *déterministe* servant de référence. Nous utilisons la fonction implémentée dans SageMath.

Mesure de l'erreur pour un nombre d'itérations fixé. Pour estimer la probabilité d'erreur de Miller–Rabin avec un nombre fixé d d'itérations (ou de bases aléatoires), on procède ainsi :

1. On fixe une taille en bits, notée **bits**.
2. On répète l'expérience un certain nombre de fois (noté N_{essais}). À chaque essai :
 - on tire au hasard un entier impair n dans l'intervalle $[1, 2^{\text{bits}})$;
 - on applique d'abord le test déterministe de référence pour vérifier que n est bien *composé* ; si n est premier, on le rejette et on tire un autre entier jusqu'à obtenir un composé ;
 - on applique ensuite à ce n le test de Miller–Rabin avec d itérations ;
 - si Miller–Rabin déclare « premier » alors que le test déterministe affirme que n est composé, on compte une erreur.
3. À la fin, on obtient un nombre total d'erreurs E_d pour N_{essais} entiers composés testés, et on estime la probabilité d'erreur en une seule expérience par la fraction

$$p_d^{(\text{emp})} \approx \frac{E_d}{N_{\text{essais}}}.$$

Moyennage sur plusieurs séries d'essais. Comme l'expérience reste aléatoire, nous répétons la procédure précédente un certain nombre de fois (noté N_{runs}) pour un même couple (bits , d). On additionne le nombre d'erreurs et le nombre total d'essais sur l'ensemble des séries, puis on calcule une probabilité d'erreur moyennée

$$\overline{p_d} \approx \frac{\text{nombre total d'erreurs}}{\text{nombre total d'entiers composés testés}}.$$

Ce moyennage permet de lisser les fluctuations dues au hasard et de comparer plus fidèlement l'expérience à la théorie.

Comparaison avec la borne théorique. Pour chaque valeur de d comprise entre 1 et une borne d_{max} , nous calculons la probabilité d'erreur empirique $\overline{p_d}$ et nous la comparons à la majoration théorique $(1/4)^d$ donnée par le théorème de Rabin. Les valeurs $\overline{p_d}$ et $(1/4)^d$ sont ensuite tracées sur le même graphe (en échelle logarithmique), ce qui permet de vérifier visuellement que la courbe empirique reste toujours nettement en dessous de la borne théorique, confirmant ainsi le comportement attendu de l'algorithme de Miller–Rabin.

2.5.2 Résultats de la simulation

L'expérience a été réalisée avec des nombres composés de 14 bits, testés sur 10^5 échantillons par itération, répétés 110 fois, ce qui donne les valeurs suivantes :

t	Erreur empirique	Borne théorique $(1/4)^t$
1	511/687500	$1/4 \approx 0,25$
2	151/2200000	$1/16 \approx 0,0625$
3	129/11000000	$1/64 \approx 0,0156$
4	19/11000000	$1/256 \approx 0,0039$
5	3/11000000	$1/1024 \approx 0,00098$
6	1/11000000	$1/4096 \approx 0,000244$
7	0	$1/16384 \approx 0,000061$

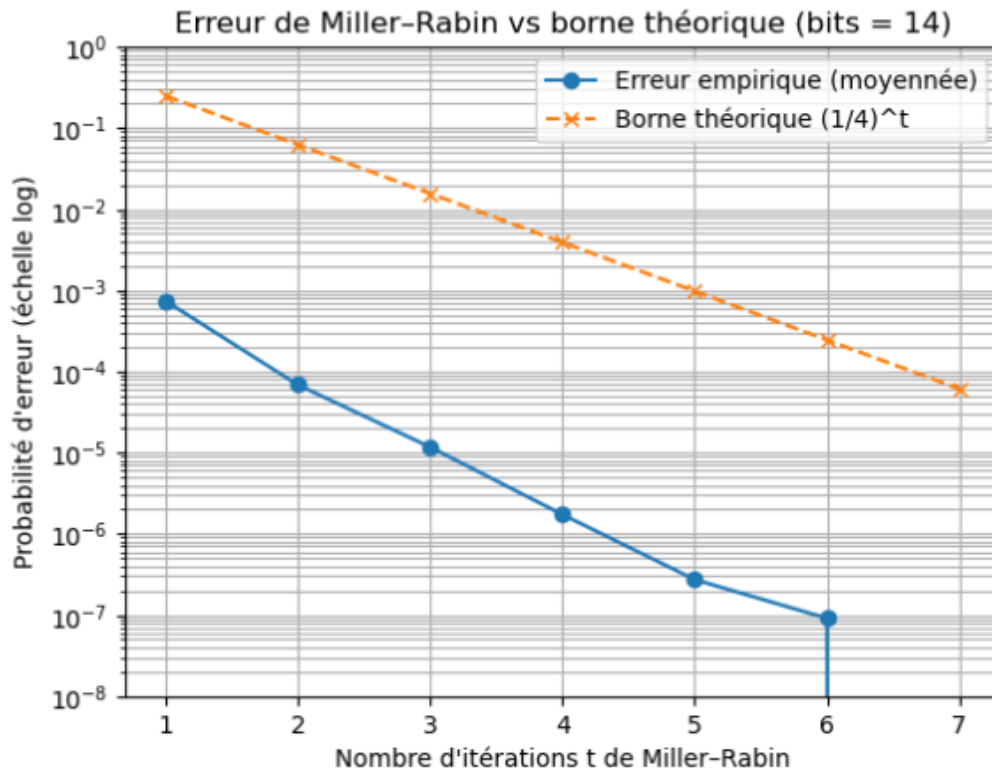


Figure 2: Comparaison entre l'erreur empirique et la borne théorique $(1/4)^t$ en échelle logarithmique.

3 Méthodes d'extraction de la racine e -ième

3.1 Méthode dichotomique (recherche binaire)

La recherche binaire consiste à trouver $\alpha = X^{1/e}$ en resserrant progressivement un intervalle contenant la racine. On initialise un encadrement (L, R) tel que $L < \alpha < R$, puis on itère en remplaçant l'une des deux bornes par le point milieu en fonction du signe de $m^e - X$.

Lemme 5 (Convergence de la méthode dichotomique). *La suite (L_k, R_k) définie par la recherche binaire satisfait*

$$R_k - L_k = \frac{R_0 - L_0}{2^k}.$$

En particulier, l'algorithme converge vers α au sens

$$\lim_{k \rightarrow \infty} L_k = \lim_{k \rightarrow \infty} R_k = \alpha.$$

Implémentation

```
1 def root_binary_search(a, e, epsilon):
2     if a < 1:
3         L, R = 0, 1
4     else:
5         L, R = 0, a
6     while R - L > epsilon:
7         m = (L + R) / 2
8         if m**e < a:
9             L = m
10        else:
11            R = m
12    return (L + R) / 2
```

3.2 Méthode de Newton

Une méthode classique, efficace et adaptée aux grands entiers, est l'algorithme de Newton appliqué à la fonction $f(x) = x^e - X$. La recherche du zéro réel $\alpha = X^{1/e}$ repose sur l'itération

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = \frac{(e-1)x_k + X/x_k^{e-1}}{e},$$

qui définit une suite d'approximation convergeant, sous des hypothèses, vers la racine réelle positive de f . La fonction f est strictement croissante et strictement convexe sur $\mathbb{R}^{*+}$, puisque $f'(x) = ex^{e-1} > 0$ et $f''(x) = e(e-1)x^{e-2} > 0$.

Ces propriétés garantissent l'unicité de la racine α et déterminent complètement la dynamique de la méthode de Newton. En particulier, la convexité implique que la tangente tracée en un point $x_k > \alpha$ coupe l'axe des abscisses en un point x_{k+1} strictement compris entre α et x_k .

Lemme 6 (Convergence monotone et encadrement). *Si l'initialisation est choisie telle que $x_0 > \alpha$, alors la suite (x_k) définie par l'itération de Newton est strictement décroissante, reste toujours au-dessus de la racine, et converge vers α . Plus précisément,*

$$\alpha < x_{k+1} < x_k \quad \text{pour tout } k \geq 0,$$

et $x_k \downarrow \alpha$.

Un second résultat classique concerne la vitesse de convergence. L'utilisation du développement de Taylor de f en x_k avec reste intégral conduit, après simplifications standard de l'analyse de Newton, à une relation quadratique sur l'erreur $\varepsilon_k = x_k - \alpha$.

Théorème 7 (Convergence quadratique). *Il existe une constante $C > 0$ telle que, pour k assez grand,*

$$|\varepsilon_{k+1}| \leq C |\varepsilon_k|^2.$$

En particulier, le nombre de chiffres exacts double à chaque itération.

Idée de la preuve. Le développement de Taylor à l'ordre 2 donne

$$f(x_k) = f(\alpha) + f'(\alpha)(x_k - \alpha) + \frac{f''(\xi_k)}{2}(x_k - \alpha)^2,$$

avec $\xi_k \in (\alpha, x_k)$. En injectant cette expression dans la formule de Newton et en utilisant $f(\alpha) = 0$, on obtient l'identité classique

$$\varepsilon_{k+1} = \frac{f''(\xi_k)}{2f'(x_k)} \varepsilon_k^2,$$

d'où la convergence quadratique en vertu de la positivité et de la continuité de f' et f'' au voisinage de α .

La question cruciale en pratique est celle de l'initialisation. Il suffit de choisir $x_0 > \alpha$; en d'autres termes, un majorant arbitraire de la racine suffit. Dans le cadre d'une extraction de racine d'un entier X de L bits, il est commode d'utiliser

$$x_0 = 2^{\lceil L/e \rceil},$$

qui satisfait toujours $x_0 > X^{1/e}$. Ce choix garantit une convergence immédiate, tout en minimisant la taille initiale des opérandes. Du point de vue arithmétique, chaque itération nécessite une puissance x_k^{e-1} , une division de grands entiers et quelques multiplications, opérations réalisables en temps quasi-linéaire en L à l'aide d'algorithmes FFT modernes. Comme la convergence est quadratique, le nombre d'itérations requis pour atteindre une précision de $B \approx L/e$ bits est logarithmique :

$$K \approx \log_2 B \approx \log_2 \left(\frac{L}{e} \right).$$

Implémentation pour grands entiers.

```

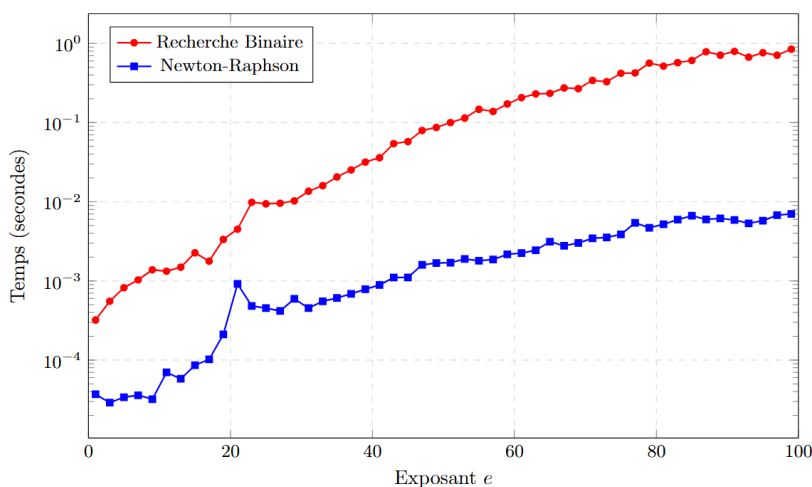
1 def nth_root_newton_integer(a, e, max_iter):
2     if a < 1:
3         return 0
4     L = a.nbits()
5     x = 2 ** (L // e)
6     for _ in range(max_iter):
7         x_prev = x
8         x = ((e-1)*x + a // (x**(e-1))) // e
9         if x == x_prev or x == x_prev - 1:
10             break
11     return x

```

3.3 Comparaison empirique des deux méthodes

Les performances relatives des méthodes dichotomique et de Newton dépendent fortement de la nature de l'exposant e et de la taille de l'entier X . La Figure 3.3 présente une comparaison des temps d'exécution pour l'extraction de racines e -ièmes d'un entier fixe de grande taille, en fonction de l'exposant e .

Comparaison des temps d'exécution: Recherche Binaire vs Newton-Raphson



On observe que la méthode de Newton surpasse systématiquement la recherche binaire, avec un avantage qui s'accroît pour les exposants moyens et élevés. Pour $e \in [20, 100]$, le gain de performance atteint un facteur de l'ordre de 10 à 100. Cette supériorité s'explique par la convergence quadratique de Newton, qui nécessite typiquement $O(\log \log X)$ itérations, contre $O(\log X)$ pour la dichotomie.

En revanche, pour les petits exposants ($e \leq 10$), l'écart reste modéré car le surcoût des opérations arithmétiques plus lourdes de Newton (puissances et divisions de grands entiers) compense partiellement l'avantage en nombre d'itérations. En pratique, la méthode de Newton demeure le choix privilégié pour l'extraction de racines d'entiers de grande taille, particulièrement dans les contextes cryptographiques où e et X peuvent être très grands.

3.4 Simulation de l'attaque de Håstad

Le code suivant implémente une simulation expérimentale de l'attaque de Håstad dans l'environnement SageMath.

```

1 def test_attack(e, r, bits = 16, method = "Newton-Raphson"):
2     primes = module_generator(r, bits)
3     m = randint(1, min(primes))
4     N = prod(primes)
5     R = crt_mod_power(m, e, primes)
6     if method == "Newton-Raphson":
7         x = nth_root_newton_integer(R, e)
8     elif method == "dichotomie":
9         x = root_binary_search(R, e)
10    else:
11        raise ExceptionName("methode non definie")
12    return (x.n() == m)

```

4 L'Attaque de Håstad avec padding linéaire

Dans le cas d'un **padding linéaire** (une forme simple de dépendance) :

$$m'_i = a_i m + b_i$$

Les messages chiffrés interceptés deviennent :

$$c_i \equiv (a_i m + b_i)^e \pmod{N_i}, \quad \text{pour } i = 1, \dots, r$$

En appliquant le CRT à ces r congruences, on obtient une unique congruence polynomiale de degré e :

$$P(x) = \sum_{i=1}^r [(a_i x + b_i)^e - c_i] M_i y_i$$

telle que $P(m) \equiv 0 \pmod{N_{tot}}$.

4.1 Outils Algébriques : Lattices et Réduction

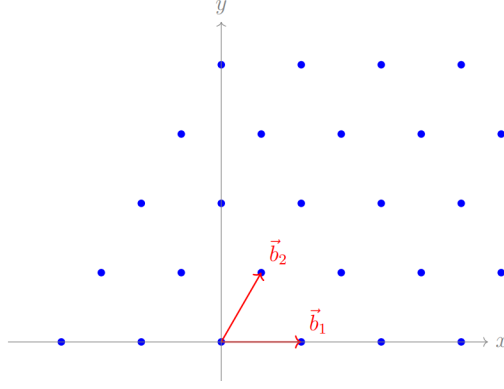
4.1.1 Définition et Propriétés des Lattices (Réseaux)

La méthode de Coppersmith repose sur la recherche de vecteurs courts dans des structures appelées lattices.

Définition 1 (Lattice). Une **lattice** (ou réseau) $\mathcal{L}$ dans $\mathbb{R}^n$ est l'ensemble de toutes les combinaisons linéaires à coefficients entiers d'un ensemble de n vecteurs linéairement indépendants $b_1, \dots, b_n \in \mathbb{R}^n$, appelés la **base** du lattice :

$$\mathcal{L} = \left\{ \sum_{i=1}^n x_i b_i : x_i \in \mathbb{Z} \right\}$$

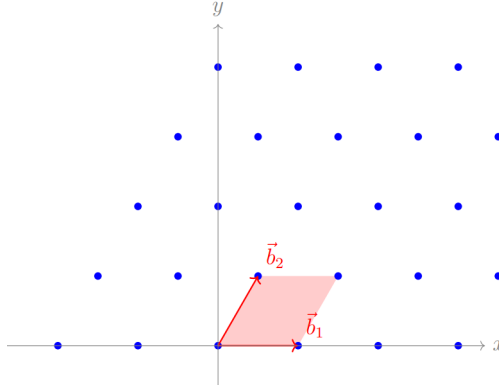
Si B est la matrice dont les colonnes (ou lignes) sont les vecteurs b_i , le lattice est aussi noté $\mathcal{L}(B)$.



Définition 2 (Déterminant et Volume Fondamental). Le **déterminant** $\det(\mathcal{L})$ d'un lattice est le volume de son **parallélépipède fondamental** $\mathcal{P}$, défini par :

$$\mathcal{P} = \left\{ \sum_{i=1}^n t_i b_i : 0 \leq t_i < 1, t_i \in \mathbb{R} \right\}$$

Si B est la matrice de la base, $\det(\mathcal{L}) = |\det(B)|$. Le déterminant est invariant quelle que soit la base choisie.



4.1.2 Le Processus d'Orthogonalisation de Gram-Schmidt

Pour mesurer la “qualité” d'une base et pour implémenter l'algorithme LLL, nous utilisons l'orthogonalisation de Gram-Schmidt.

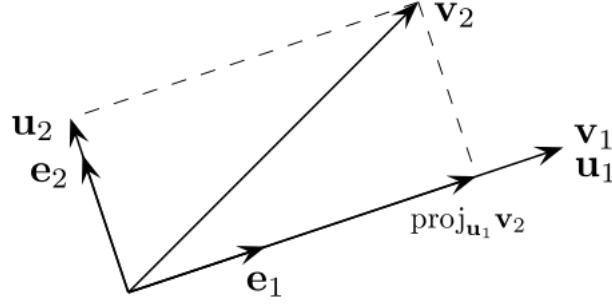
Définition 3 (Base Orthogonale de Gram-Schmidt). Étant donné une base $b_1, \dots, b_n$ du lattice $\mathcal{L}$, le processus de Gram-Schmidt calcule une base **orthogonale** $b_1^*, \dots, b_n^*$. Les vecteurs b_i^* et les coefficients $\mu_{i,j}$ (les projections) sont définis récursivement :

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{i,j} b_j^*$$

$$\mu_{i,j} = \frac{\langle b_i, b_j^* \rangle}{\|b_j^*\|^2}$$

Preuve de l'Orthogonalité. Par construction, pour $k < i$, $\langle b_i^*, b_k^* \rangle = \langle b_i - \sum_{j=1}^{i-1} \mu_{i,j} b_j^*, b_k^* \rangle$. Par la définition de $\mu_{i,k}$, on a $\langle b_i, b_k^* \rangle = \mu_{i,k} \|b_k^*\|^2$. Donc :

$$\langle b_i^*, b_k^* \rangle = \mu_{i,k} \|b_k^*\|^2 - \sum_{j=1}^{i-1} \mu_{i,j} \langle b_j^*, b_k^* \rangle$$



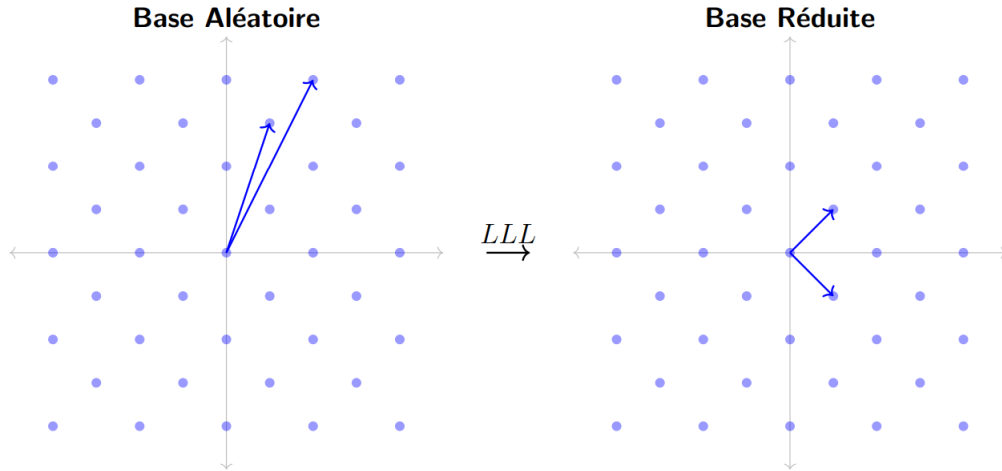
Si l'on suppose que $\{b_1^*, \dots, b_{i-1}^*\}$ est déjà orthogonal, le terme $\langle b_j^*, b_k^* \rangle$ est nul sauf si $j = k$.

$$\langle b_i^*, b_k^* \rangle = \mu_{i,k} \|b_k^*\|^2 - \mu_{i,k} \|b_k^*\|^2 = 0$$

Ceci démontre l'orthogonalité par récurrence. □

4.1.3 L'Algorithme LLL (Lenstra-Lenstra-Lovász)

L'objectif de LLL [4] est de transformer une base arbitraire $B = (b_1, \dots, b_n)$ en une base $B' = (b'_1, \dots, b'_n)$ dite **réduite** ou "presque orthogonale" dont les vecteurs sont significativement plus courts. Cette étape est essentielle car elle facilite la résolution du problème du vecteur le plus court (SVP), une tâche NP-difficile en général. LLL est un algorithme en temps polynomial qui fournit une approximation du vecteur le plus court.



Définition 4 (Base Réduite LLL). Une base $b_1, \dots, b_n$ est δ -LLL réduite (avec $\frac{1}{4} < \delta < 1$) si elle satisfait les deux conditions suivantes, en utilisant les vecteurs orthogonaux de Gram-Schmidt b_i^* et les coefficients de projection $\mu_{i,j}$:

1. **Condition de Trivialité (Taille Réduite)** : Pour tout $1 \leq j < i \leq n$, le coefficient de projection $\mu_{i,j}$ est borné :

$$|\mu_{i,j}| \leq \frac{1}{2}$$

Cette condition est maintenue par des réductions successives (similaires à l'algorithme d'Euclide), assurant que la partie non-orthogonale de b_i par rapport aux vecteurs précédents b_j est minimale.

2. **Condition d'Interchange (Lovász)** : Pour tout $i = 2, \dots, n$, la longueur du vecteur orthogonal b_i^* est liée à la longueur du vecteur précédent b_{i-1}^* :

$$\|b_i^* + \mu_{i,i-1} b_{i-1}^*\|^2 \geq \delta \|b_{i-1}^*\|^2$$

Cette condition est la plus importante et impose une certaine **quasi-orthogonalité** à la base. Si elle n'est pas satisfaite, les vecteurs b_i et b_{i-1} sont échangés pour tenter de raccourcir la base.

Théorème 8 (Borne sur le Premier Vecteur d'une Base LLL). *Si $b_1, \dots, b_n$ est une base δ -LLL réduite d'un lattice $\mathcal{L}$, alors le premier vecteur b_1 est court :*

$$\|b_1\| \leq \left(\frac{1}{\delta - 1/4} \right)^{\frac{n-1}{2}} \det(\mathcal{L})^{1/n}$$

En choisissant la valeur courante $\delta = 3/4$, cette borne garantit que le vecteur b_1 est à une distance au plus exponentielle du vecteur le plus court du lattice :

$$\|b_1\| \leq 2^{\frac{n-1}{2}} \det(\mathcal{L})^{1/n}$$

4.2 L'Algorithme de Coppersmith et le Théorème de Howgrave-Graham

4.2.1 Énoncé du Théorème de Coppersmith (Racines Petites Modulo N)

L'algorithme de Coppersmith repose sur un théorème fondamental qui permet de trouver efficacement des racines entières d'un polynôme qui sont petites par rapport au module.

Théorème 9 (Théorème de Coppersmith, 1997). *Soit N un entier et $f \in \mathbb{Z}[x]$ un polynôme unitaire de degré d sur les entiers. Soit $X = N^{1/d-\epsilon}$ pour un $0 < \epsilon < 1/d$. Alors, étant donné (N, f) , l'attaquant peut trouver **efficacement** (en temps polynomial) tous les entiers $x_0 < X$ satisfaisant*

$$f(x_0) \equiv 0 \pmod{N}$$

Ce théorème fondamental affirme qu'il existe un algorithme capable de trouver **toutes les racines de f modulo N qui sont plus petites que la borne $X = N^{1/d-\epsilon}$** . L'idée principale de ce théorème est donc la capacité de trouver de **petites racines** de polynômes modulo un composite N . Plus la borne X est petite, plus son temps d'exécution diminue.

4.2.2 Contexte et Transformation du Problème

L'objectif de l'algorithme de Coppersmith, dans le contexte de l'attaque de Håstad, est de trouver la petite racine entière m du polynôme $P(x)$ de degré $d = e$ (l'exposant RSA) tel que $P(m) \equiv 0 \pmod{N_{tot}}$. Nous cherchons m tel que $|m| < X$ où la borne de succès théorique est $X = N_{tot}^{1/d-\epsilon}$.

La stratégie est de construire un lattice $\mathcal{L}$ qui, après réduction par LLL, contient un vecteur court correspondant à un nouveau polynôme $Q(x)$ tel que $Q(m)$ est nul **non seulement modulo N_{tot} , mais aussi sur les entiers $\mathbb{Z}$** .

4.2.3 Le Théorème de Howgrave-Graham (Conversion Modulo $\rightarrow$ Entier)

La réduction du lattice par LLL fournit un vecteur court v' , qui correspond à un nouveau polynôme $Q(x)$. Le **Lemme de Howgrave-Graham** est la clé pour montrer que $Q(m)$ est nul **non seulement modulo N , mais aussi sur les entiers $\mathbb{Z}$** .

Théorème 10 (Lemme de Howgrave-Graham, 1997). *Soit $P(x) \in \mathbb{Z}[x]$ un polynôme de degré D . Soit X une borne. Si un entier m vérifie :*

1. $P(m) \equiv 0 \pmod{N}$
2. La norme euclidienne des coefficients du polynôme $P(xX)$ est bornée par :

$$\|P(xX)\| = \sqrt{\sum_{i=0}^D (\text{Coeff}(x^i \text{ dans } P(xX)))^2} < \frac{N}{\sqrt{D+1}}$$

Alors, $P(m) = 0$ sur les entiers $\mathbb{Z}$.

Remarque 2. L'algorithme de Coppersmith est donc un processus en trois étapes :

1. **Construction** : Définir la matrice de lattice B à partir de $P(x)$ et de la borne X . (Les détails de la construction de la matrice B sont donnés en Annexe.)
2. **Réduction** : Appliquer l'algorithme LLL à B pour trouver le vecteur court v' qui donne $Q(x)$.
3. **Résolution** : Résoudre $Q(m) = 0$ sur les entiers pour trouver m .

4.3 Simulation de l'attaque de Hastad avec padding linéaire

Le code suivant implémente une simulation expérimentale de l'attaque de Hastad avec padding linéaire dans l'environnement SageMath. Il génère plusieurs modules RSA, construit le polynôme global via le théorème des restes chinois (CRT) et applique la méthode de Coppersmith pour retrouver le message caché.

```

1 def test_attack(e, r, epsilon = 1/(4*e), bits = 16):
2     moduli = module_generator(r, bits)      # list of N_i
3     N_total = prod(moduli)                 # product of all N_i
4     X = floor(N_total^(1/e - epsilon))     # m < X
5     m = randint(1, min(moduli))           # Hidden RSA message
6
7     R = PolynomialRing(Zmod(N_total), 'x')
8     x = R.gen()
9     P = R(0)
10
11     for i in range(r):
12         N_i = moduli[i]
13         a_i = randint(1, 100)
14         b_i = randint(1, 1000)
15
16         m_i = a_i * m + b_i
17         c_i = power_mod(m_i, e, N_i)
18
19         M_i = N_total // N_i               # CRT partial modulus
20         inv_M_i = power_mod(M_i, -1, N_i)  # inverse of M_i mod N_i
21         W_i = M_i * inv_M_i               # CRT weight
22
23         local_poly = (a_i * x + b_i)^e - c_i
24         P += local_poly * W_i
25
26     P_monic = P.monic() # Required for small_roots()
27     racines = P_monic.small_roots(X=X, epsilon=epsilon)
28
29     return (len(racines) == 1 and racines[0] == m)

```

4.4 Analyse de la borne d'attaque

Le théorème de Coppersmith impose que pour trouver les racines d'un polynôme modulaire, la racine recherchée m doit satisfaire $m < X$ où la borne X est définie par :

$$X = N_{\text{total}}^{\frac{1}{e} - \epsilon}$$

Par ailleurs, le protocole RSA impose que le message m vérifie :

$$m < \min_{i \in [1, r]} N_i = N$$

où N désigne le plus petit des modules RSA utilisés.

Pour que l'attaque réussisse, il faut que la borne de Coppersmith X soit au moins aussi grande que la contrainte RSA N , c'est-à-dire $X \geq N$. En substituant l'expression de X , nous obtenons :

$$N_{\text{total}}^{\frac{1}{e} - \epsilon} \geq N$$

Sachant que $N_{\text{total}} = \prod_{i=1}^r N_i$ et en approximant tous les modules par leur minimum N (hypothèse conservative), nous avons $(N^r)^{\frac{1}{e} - \epsilon} \geq N$, soit $N^{r(\frac{1}{e} - \epsilon)} \geq N$. En simplifiant (pour $N > 1$), cela donne $r(\frac{1}{e} - \epsilon) \geq 1$, d'où la condition sur le nombre minimal de destinataires :

$$r \geq \frac{e}{1 - e\epsilon}$$

Cette borne reste très large et peu précise en pratique. En effet, elle suppose une égalité des modules, ce qui n'est généralement pas le cas. Il est donc nécessaire de raffiner cette analyse pour déterminer précisément où l'attaque réussit en pratique.

Pour mieux comprendre le comportement réel de l'attaque, introduisons une borne plus fine β qui caractérise la distribution effective des messages attaquables. Considérons que les messages m sont uniformément distribués dans $[1, N]$. Nous cherchons à exprimer β en fonction des paramètres de l'attaque. Posons $\varepsilon = \frac{1}{k \cdot e}$ avec $k > 1$, où k est un facteur d'ajustement. Alors $\beta = f(r, e, k)$. Cette fonction $\beta(r, e, k)$ représente la proportion de l'espace des messages qui peuvent être récupérés avec succès par l'attaque.

Pour quantifier la réussite de l'attaque, nous définissons la probabilité de succès conditionnelle :

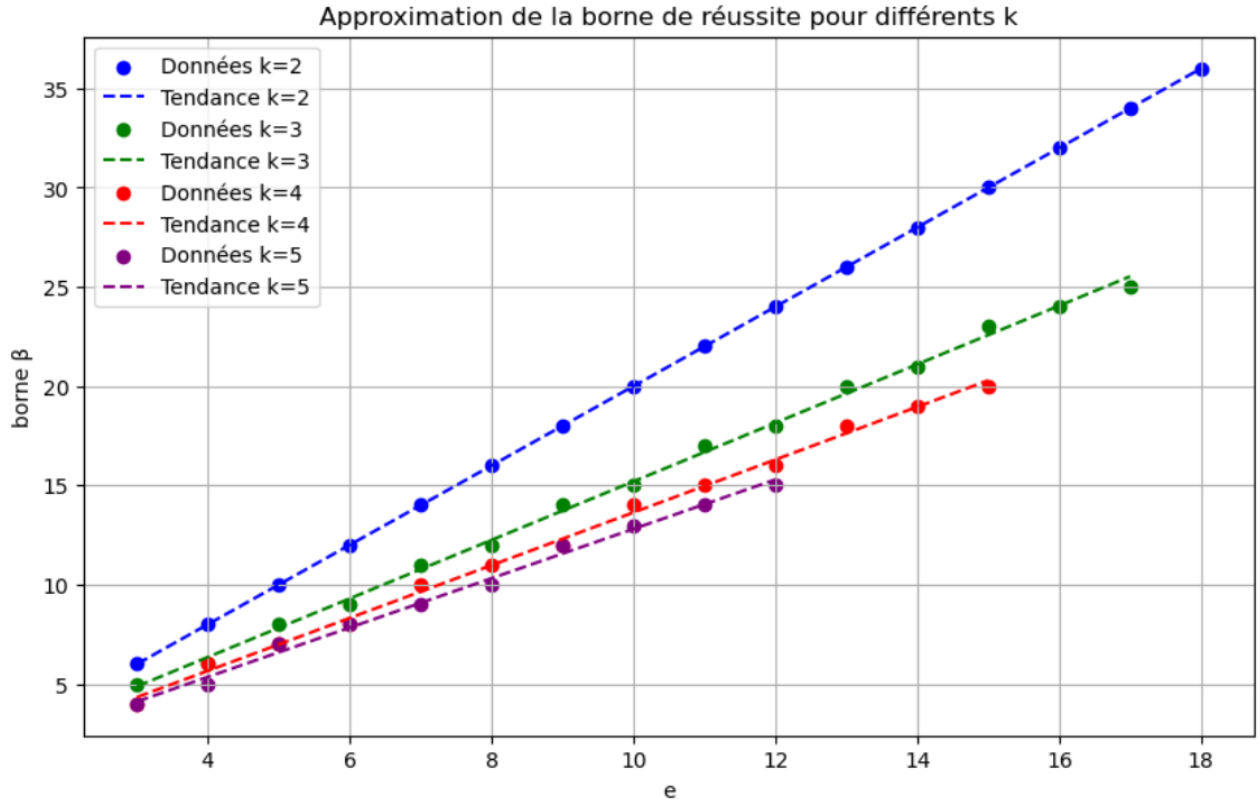
$$P(\text{succès} \mid r, e, k) = \frac{\text{count}(\text{succès})}{\text{total count}}$$

où $\text{count}(\text{succès})$ est le nombre de messages m récupérés avec succès et total count est le nombre total de tentatives. La valeur optimale de β peut être déterminée empiriquement comme :

$$\beta(e, k) = \min_r \{P(\text{succès} \mid r, e, k) = 1\}$$

C'est-à-dire, β correspond au nombre minimal de destinataires r pour lequel l'attaque réussit avec certitude (probabilité 1). Cette approche empirique permet d'affiner la borne théorique large obtenue précédemment, de déterminer précisément le seuil de réussite pratique de l'attaque, et de comprendre l'influence réelle de chaque paramètre (r, e, k) sur l'efficacité.

4.5 Résultats expérimentaux et analyse



Après simulation exhaustive de l'attaque pour différentes valeurs de (e, k) , nous avons déterminé empiriquement les valeurs de $\beta(e, k) = \min_r \{P(\text{succès} \mid r, e, k) = 1\}$ pour chaque configuration. Les données révèlent une relation linéaire entre β et e pour chaque valeur fixée de k , que nous pouvons modéliser par :

$$\beta(e, k) = a(k) \cdot e + b(k)$$

Les régressions linéaires obtenues sont :

$$\begin{aligned} k = 2 : \quad \beta_2 &\approx 2.000 \cdot e + 0.000 \\ k = 3 : \quad \beta_3 &\approx 1.475 \cdot e + 0.450 \\ k = 4 : \quad \beta_4 &\approx 1.330 \cdot e + 0.341 \\ k = 5 : \quad \beta_5 &\approx 1.242 \cdot e + 0.382 \end{aligned}$$

Ces résultats permettent plusieurs observations critiques. Premièrement, le coefficient $a(k)$ décroît strictement avec k , ce qui signifie que pour des valeurs élevées de e , un k plus grand permet d'obtenir une borne β plus petite, rendant l'attaque plus efficace sur une plus grande proportion de messages. En effet, pour $k = 2$, nous avons $\beta \approx 2e$, tandis que pour $k = 5$, $\beta \approx 1.24e$, soit une réduction significative.

Deuxièmement, bien que l'augmentation de k améliore la borne théorique, elle entraîne une augmentation considérable de la complexité algorithmique de la réduction de réseau LLL, rendant le calcul pratique plus coûteux en temps et en ressources. Il existe donc un compromis entre l'efficacité théorique et la faisabilité pratique.

Troisièmement, pour un e élevé, la borne β croît linéairement, ce qui implique que le nombre minimal de destinataires r nécessaire pour garantir le succès de l'attaque augmente également. Ainsi, les systèmes utilisant des exposants publics élevés bénéficient d'une meilleure résistance naturelle contre l'attaque de Håstad, car celle-ci nécessite davantage d'interceptions de chiffrés du même message.

Quatrièmement, le terme constant $b(k)$ reste relativement stable et faible pour toutes les valeurs de k testées, ce qui suggère que l'effet dominant sur β provient du terme linéaire $a(k) \cdot e$. Enfin, ces résultats empiriques confirment que la borne théorique initiale $r \geq \frac{e}{1-e\epsilon}$ était effectivement trop large, et que l'approche expérimentale fournit une caractérisation bien plus précise du seuil de réussite réel de l'attaque.

4.6 Synthèse et Résultat Final

4.6.1 L'Enchaînement de l'Attaque Coppersmith-Håstad

L'efficacité de la méthode de Coppersmith pour briser l'attaque de Håstad, même avec un padding linéaire, repose sur la chaîne d'arguments suivante :

1. **Problème Initial (Håstad)** : Chiffrement de m avec e petit et padding linéaire $m'_i = a_i m + b_i$ pour r modules distincts N_i .
2. **Transformation (CRT)** : $P(m) \equiv 0 \pmod{N_{tot}}$, où $P(x)$ est de degré $d = e$.
3. **Vulnérabilité (Petite Racine)** : m est une petite racine, vérifiant $|m| < N_{tot}^{1/e}$.
4. **Résolution (Lattice)** : Construction de $\mathcal{L}$ basé sur $P(x)$, échelonné par $X = N_{tot}^{1/e-\epsilon}$.
5. **Déchiffrement (LLL et Howgrave-Graham)** : LLL trouve $Q(x)$ (vecteur court) et le Lemme de Howgrave-Graham garantit $Q(m) = 0$ sur les entiers $\mathbb{Z}$.

Une fois le polynôme $Q(x)$ obtenu, la racine m est trouvée rapidement :

$$m = \text{Racine Entière}(Q(x))$$

4.6.2 Conclusion sur l'Insécurité du Padding Déterministe

L'attaque de Håstad, généralisée par Coppersmith, a démontré que tout schéma de padding $\mathcal{P}(m)$ qui maintient une **relation polynomiale de bas degré** avec le message secret m est vulnérable si un attaquant peut intercepter suffisamment de messages chiffrés. Le message de cette cryptanalyse est clair : la sécurité de RSA doit reposer sur l'aléatoire et l'imprévisibilité de l'étape de padding.

4.6.3 Contre-Mesures et Sécurité RSA : Le rôle d’OAEP

Pour garantir la sécurité contre les attaques de diffusion (Broadcast Attacks), il est impératif d’utiliser un schéma de padding qui rompt le lien polynomial entre le message clair m et le bloc chiffré m' .

Définition 5 (Optimal Asymmetric Encryption Padding (OAEP)). *Le schéma OAEP [6], standardisé par PKCS#1 v2.0, utilise une fonction de hachage et une valeur aléatoire (seed ou salt) pour construire le bloc chiffré.*

$$m' = (m \parallel \text{padding}) \oplus G(r) \parallel r \oplus H((m \parallel \text{padding}) \oplus G(r))$$

où G et H sont des fonctions de hachage (par exemple SHA-256), et r est une chaîne aléatoire.

L’incorporation de la chaîne aléatoire r et des fonctions de hachage garantit que :

1. **Non-Déterminisme** : Pour le même message m , le bloc chiffré m' est différent à chaque exécution.
2. **Absence de Lien Polynomial** : La nature chaotique des fonctions de hachage rend impossible l’établissement d’une relation polynomiale simple, rendant les méthodes de Coppersmith-Håstad inutilisables.

L’utilisation de **RSA-OAEP** est donc la norme pour un chiffrement sûr.

5 Conclusion Générale

Ce rapport a mis en évidence une vulnérabilité fondamentale du schéma RSA dans le contexte de la diffusion large : l’utilisation d’un exposant public commun pour chiffrer un même message vers plusieurs destinataires permet à un attaquant d’inverser le chiffrement sans connaissance des clés privées. Nous avons démontré que cette attaque devient réalisable de manière déterministe dès que le nombre de destinataires interceptés atteint ou dépasse l’exposant public ($r \geq e$), réduisant ainsi le problème cryptographique à un simple calcul de racine entière.

L’analyse théorique et expérimentale du test de Miller-Rabin a confirmé son efficacité pour la génération de grands nombres premiers cryptographiques. Nos simulations ont validé la borne d’erreur théorique 4^{-t} et montré que le nombre moyen d’essais nécessaires pour obtenir un nombre premier de ℓ bits suit fidèlement les prédictions du théorème des nombres premiers, avec environ $\frac{\ln(2^\ell)}{2}$ tirages en moyenne. Cette convergence entre théorie et pratique justifie pleinement l’utilisation de Miller-Rabin dans les implémentations RSA modernes.

L’extension de l’attaque par les méthodes de Håstad et Coppersmith révèle une portée encore plus large de cette vulnérabilité. Même en présence d’un padding linéaire, censé différencier les messages, la structure polynomiale résiduelle permet une récupération efficace du message original via la réduction de réseaux (algorithme LLL). Nos analyses expérimentales ont déterminé les bornes pratiques d’attaque : pour un paramètre $\varepsilon = \frac{1}{ke}$ avec $k \geq 2$, le nombre critique de destinataires suit approximativement $r_{\text{critique}} \approx a(k) \cdot e$ où $a(k)$ décroît avec k (de $a(2) \approx 2,0$ à $a(5) \approx 1,24$).

Ces résultats soulignent l’importance critique du padding aléatoire dans les implémentations RSA. Le schéma OAEP (Optimal Asymmetric Encryption Padding), en incorporant des valeurs aléatoires et des fonctions de hachage cryptographiques, rompt définitivement la relation polynomiale entre messages et chiffrés, rendant les méthodes de Coppersmith-Håstad inapplicables. La leçon fondamentale de cette étude est claire : la sécurité de RSA ne repose pas uniquement sur la difficulté mathématique de la factorisation, mais également sur l’imprévisibilité introduite par un padding robuste. Toute implémentation RSA moderne doit impérativement utiliser RSA-OAEP ou un schéma équivalent pour garantir une sécurité effective face aux attaques de diffusion.

Enfin, ce travail illustre la puissance des techniques de cryptanalyse algébrique et géométrique (réseaux euclidiens) dans l’analyse de sécurité des primitives cryptographiques, démontrant qu’une conception cryptographique sûre nécessite une compréhension profonde non seulement des hypothèses de sécurité sous-jacentes, mais aussi des interactions subtiles entre les différentes composantes du schéma.

References

- [1] Rivest, R. L., Shamir, A., Adleman, L. *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. Communications of the ACM, 21(2):120–126, 1978. DOI: 10.1145/359340.359342
- [2] Håstad, J. *On Using RSA with Low Exponent in a Public Key Network*. CRYPTO '85, LNCS 218:403–408, Springer, 1986. DOI: 10.1007/3-540-39568-7_29
- [3] Coppersmith, D. *Small Solutions to Polynomial Equations, and Low Exponent RSA Vulnerabilities*. Journal of Cryptology, 10(4):233–260, 1997. DOI: 10.1007/s001459900020
- [4] Lenstra, A. K., Lenstra, H. W. Jr., Lovász, L. *Factoring Polynomials with Rational Coefficients*. Mathematische Annalen, 261(4):515–534, 1982. DOI: 10.1007/BF01457454
- [5] Howgrave-Graham, N. *Finding Small Roots of Univariate Modular Equations Revisited*. IMA Conference on Cryptography and Coding, LNCS 1355:131–142, Springer, 1997. DOI: 10.1007/BFb0028701
- [6] Bellare, M., Rogaway, P. *Optimal Asymmetric Encryption*. EUROCRYPT '94, LNCS 950:92–111, Springer, 1994. DOI: 10.1007/BFb0053428
- [7] Golub, G. H., Van Loan, C. F. *Matrix Computations*. 3rd edition, Johns Hopkins University Press, 1996. Lien Archive
- [8] RSA Laboratories. *PKCS #1 v2.1: RSA Cryptography Standard*. RFC 3447, 2003. Lien IETF (RFC)

A Annexes

A.1 Construction du Lattice Spécifique à la Résolution Polynomiale

Soit N le module (dans notre cas $N = N_{\text{tot}}$), $P(x) \in \mathbb{Z}[x]$ un polynôme de degré d , et X une borne telle que la racine recherchée m vérifie $|m| < X$. Nous introduisons deux paramètres entiers positifs h (puissance du module) et k (nombre de décalages), choisis conformément à la méthode de Coppersmith–Howgrave-Graham.

A.1.1 Définition des Polynômes Générateurs

Le lattice $\mathcal{L}$ est engendré par les vecteurs de coefficients de polynômes construits de manière à s’annuler modulo N^h en $x = m$.

1. **Polynômes de type “shift” modulo N^h** : Pour $0 \leq i \leq k-1$ et $0 \leq j \leq h-1$, on définit

$$p_{i,j}(x) = x^i \cdot N^{h-1-j} \cdot P(x)^j.$$

Si $P(m) \equiv 0 \pmod{N}$, alors $p_{i,j}(m) \equiv 0 \pmod{N^h}$.

2. **Polynômes de type “module”** : Pour $0 \leq i \leq k-1$, on ajoute

$$p_{i,h}(x) = x^i \cdot N^h.$$

Ces polynômes garantissent que toute combinaison linéaire entière des générateurs reste congrue à zéro modulo N^h .

Remarque 3 (Substitution $x \mapsto xX$). *Afin d’appliquer le lemme de Howgrave-Graham, on effectue la substitution $x \mapsto xX$ dans tous les polynômes générateurs. Pour un monôme $a_\ell x^\ell$, on obtient*

$$a_\ell (xX)^\ell = a_\ell X^\ell x^\ell.$$

Cette substitution :

- normalise la racine m en $y = m/X$ avec $|y| < 1$;
- équilibre les contributions des coefficients dans la norme euclidienne du vecteur de coefficients, condition essentielle pour que la borne de Howgrave-Graham soit applicable.

A.1.2 Dimension et Structure du Lattice

Le lattice $\mathcal{L}$ est engendré par les vecteurs de coefficients des polynômes $p_{i,j}(xX)$.

Définition 6 (Degré maximal). *Le degré de $p_{i,j}(x)$ est $i + jd$. Le degré maximal est atteint pour $i = k-1$ et $j = h$:*

$$D = (k-1) + hd.$$

Remarque 4 (Dimension de la base). • *Le nombre total de polynômes générateurs est*

$$T = k(h+1).$$

- *Le nombre de monômes distincts est $D+1$.*

En pratique, on choisit les paramètres h et k de sorte que $T \geq D+1$, puis on considère un sous-lattice de rang maximal $(D+1)$. Il n’est pas nécessaire que la matrice initiale soit carrée : le déterminant pertinent est celui du lattice engendré.

Chaque ligne de la matrice de base B correspond au vecteur de coefficients d’un polynôme $p_{i,j}(xX)$, ordonné selon les puissances croissantes de x .

A.1.3 Borne sur le Déterminant et Condition de Succès

Esquisse de la borne du déterminant. Après la substitution $x \mapsto xX$, chaque coefficient est multiplié par une puissance de X dépendant du degré du monôme. Le déterminant du lattice $\mathcal{L}$ ne se calcule pas exactement, mais peut être majoré par un produit de deux contributions principales :

- une contribution en N , issue des facteurs N^{h-1-j} et N^h ;
- une contribution en X , issue de la mise à l'échelle des monômes.

On obtient une borne de la forme

$$\det(\mathcal{L}) \leq N^{\Theta(h^2k)} \cdot X^{\Theta(D^2)}.$$

Le théorème de réduction LLL garantit l'existence d'un vecteur non nul de norme suffisamment petite. En appliquant le lemme de Howgrave-Graham, on obtient un polynôme $Q(x)$ tel que

$$Q(m) = 0 \quad \text{sur } \mathbb{Z}.$$

L'optimisation des paramètres h et k conduit à la condition asymptotique classique de succès :

$$\boxed{X \leq N^{\frac{1}{d}-\varepsilon}} \quad \text{pour un } \varepsilon > 0.$$

□

A.1.4 Illustration : Exemple Simplifié

Considérons un polynôme linéaire ($d = 1$) et les paramètres $h = 2$, $k = 2$.

Polynômes générateurs (avant substitution)

- $j = 0$:

$$p_{0,0}(x) = N, \quad p_{1,0}(x) = Nx$$

- $j = 1$:

$$p_{0,1}(x) = P(x), \quad p_{1,1}(x) = xP(x)$$

- $j = 2$:

$$p_{0,2}(x) = N^2, \quad p_{1,2}(x) = N^2x$$

On a $T = 6$ polynômes et un degré maximal

$$D = (k - 1) + hd = 3.$$

Après substitution $x \mapsto xX$ et en ordonnant les monômes de x^0 à x^3 , la matrice de base B (avec $P(x) = P_0 + P_1x$) s'écrit :

$$B = \begin{pmatrix} N & 0 & 0 & 0 \\ 0 & NX & 0 & 0 \\ P_0 & P_1X & 0 & 0 \\ 0 & P_0X & P_1X^2 & 0 \\ N^2 & 0 & 0 & 0 \\ 0 & N^2X & 0 & 0 \end{pmatrix}$$

Cette matrice illustre la structure *approximativement triangulaire par blocs* du lattice, typique des constructions de Coppersmith.